

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**
Высшая школа электроники и компьютерных наук
Кафедра системного программирования

ДОПУСТИТЬ К ЗАЩИТЕ

Заведующий кафедрой, д.ф.-м.н.,
профессор

_____ Л.Б. Соколинский
« ____ » _____ 2025 г.

Реализация веб-приложения для детекции дорожных объектов

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ЮУрГУ – 02.03.02.2025.308-007.ВКР

Научный руководитель,
ст. преподаватель кафедры СП
_____ Н.С. Силкина

Автор работы,
студент группы КЭ-401
_____ Д.А. Иванов

Ученый секретарь
(нормоконтролер)
_____ И.Д. Володченко
« ____ » _____ 2025 г.

Челябинск, 2025 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**
Высшая школа электроники и компьютерных наук
Кафедра системного программирования

УТВЕРЖДАЮ
Зав. кафедрой СП
_____ Л.Б. Соколинский
27.01.2025 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы бакалавра
студенту группы КЭ-401

Иванову Дмитрию Александровичу,
обучающемуся по направлению

02.03.02 «Фундаментальная информатика и информационные технологии»

1. **Тема работы** (утверждена приказом ректора от 21.04.2025 г. № 648-13/12)
Реализация веб-приложения для детекции дорожных объектов.
2. **Срок сдачи студентом законченной работы:** 02.06.2025 г.
3. **Исходные данные к работе**
 - 3.1. Курс Samsung «Искусственный интеллект» [Электронный ресурс] URL: <https://innovationcampus.ru/ai/> (дата обращения: 27.01.2025 г.)
 - 3.2. Набор данных «Road DETECTION - IMGs & Labels» [Электронный ресурс] URL: <https://www.kaggle.com/datasets/princekhunt19/road-detection-imgs-and-labels> (дата обращения: 27.01.2025 г.)
 - 3.3. Документация Ultralytics YOLO [Электронный ресурс] URL: <https://docs.ultralytics.com/> (дата обращения: 27.01.2025 г.).
4. **Перечень подлежащих разработке вопросов**
 - 4.1. Провести анализ предметной области.
 - 4.2. Подготовить набор данных.
 - 4.3. Обучить и протестировать нейросетевые модели.
 - 4.4. Спроектировать и реализовать веб-приложение.
 - 4.5. Провести тестирование веб-приложения.
5. **Дата выдачи задания:** 27.01.2025 г.

Научный руководитель,
ст. преподаватель кафедры СП

Н.С. Силкина

Задание принял к исполнению

Д.А. Иванов

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	4
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	7
1.1. Обзор аналогичных проектов	7
1.2. Анализ существующих решений.....	8
1.3. Метрики качества.....	10
2. НЕЙРОСЕТЕВАЯ МОДЕЛЬ ДЕТЕКЦИИ ОБЪЕКТОВ.....	14
2.1. Обучающий набор данных.....	14
2.2. Обучение нейросетевых моделей.....	16
2.3. Эксперименты	20
3. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ	26
3.1. Проектирование	26
3.2. Серверная часть приложения.....	28
3.3. Клиентская часть приложения.....	29
3.4. Тестирование приложения	31
ЗАКЛЮЧЕНИЕ	32
ЛИТЕРАТУРА.....	33

ВВЕДЕНИЕ

Актуальность

Искусственный интеллект (ИИ) и нейросети в настоящее время являются одним из популярных быстроразвивающихся направлений в области информационных технологий.

По данным на начало 2025 года, объем глобального рынка искусственного интеллекта составляет 757,58 млрд долларов. К 2034 году этот показатель, по прогнозам, достигнет 3 680,47 млрд долларов [7].

Они широко применяются во многих отраслях, включая обработку естественного языка, анализ изображений и видео, прогнозирование поведения потребителей и автоматизацию процессов.

Обработкой и анализом изображений занимается компьютерное зрение – область ИИ, создающая программы и системы, позволяющие компьютерам воспринимать визуальную информацию.

Детекция объектов – это одна из задач компьютерного зрения, которая заключается в обнаружении и локализации объектов заданных классов на изображении или видео [6]. Работа детекции представлена на рисунке 1.

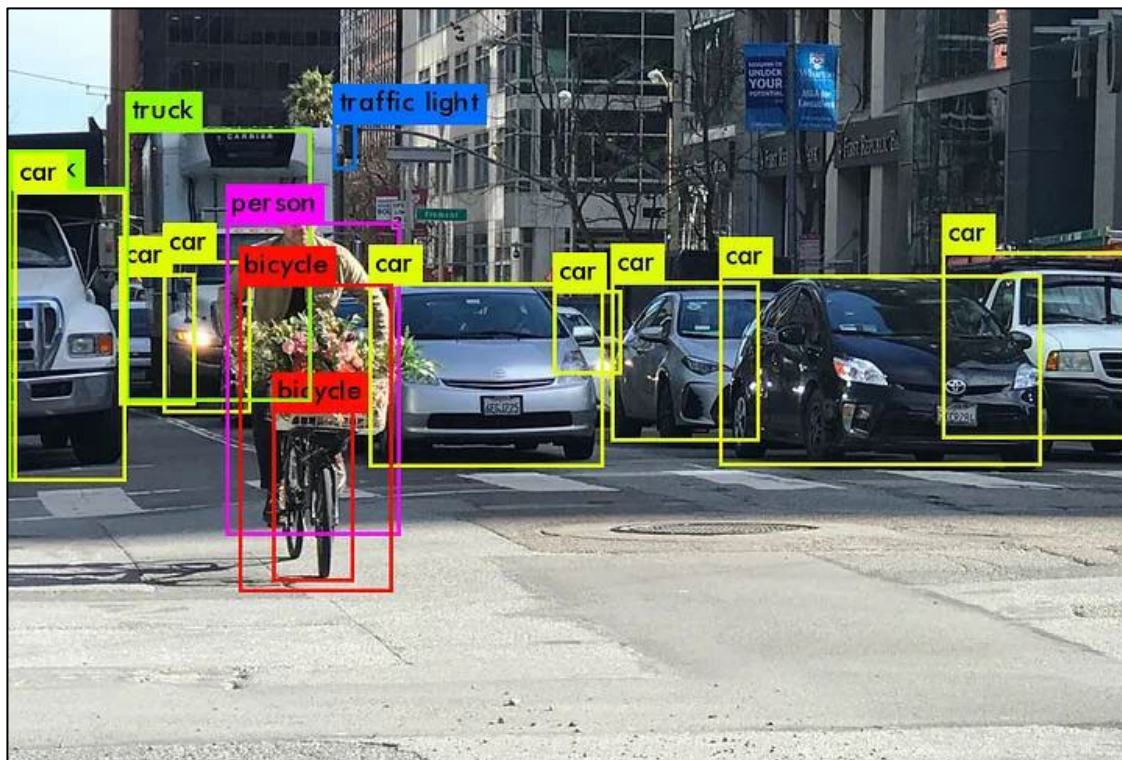


Рисунок 1 – Демонстрация детекции

Разработка нейросетей для задач детекции, является актуальным направлением в связи с широким спектром применений и потенциальных выгод. Ниже приведены некоторые особенности и способы применения.

1. Нейросети способны выявлять различные зависимости, обнаруживать элементы, которых раньше не было в сети, и изучать закономерности [1].

2. Медицинские изображения могут быть проанализированы для обнаружения опухолей или других патологий с высокой точностью благодаря алгоритмам детекции.

3. Детекция дорожных объектов, таких как автомобили, пешеходы и дорожные знаки, играет важную роль в обеспечении безопасности на дорогах, особенно при внедрении в системы помощи водителю (ADAS) [4].

4. Современные типы архитектуры, такие как YOLO, быстро и эффективно работают на больших наборах данных, что критически важно для безопасности на дороге [4].

5. Нейросети могут демонстрировать хорошую точность при работе с изображениями различного качества и в различных условиях освещения, что важно для работы в разных погодных условиях и в разное время суток [4].

6. Разработка нейросетей для детекции объектов является важной для автономных транспортных средств, медицины, систем видеонаблюдения, и дополненной реальности [2].

7. Искусственный интеллект применяется для фиксации нарушений ПДД, например, превышения скорости, с использованием дорожных камер и нейросетей [5].

8. Искусственный интеллект используется для прогнозирования чрезвычайных ситуаций и моделирования сценариев реагирования на них [5].

Постановка задачи

Целью выпускной квалификационной работы является реализация веб-приложения для детекции дорожных объектов. Также в ходе работы будет обучена нейросетевая модель, на основе которой будет работать веб-приложение. Для достижения поставленной цели необходимо решить следующие задачи:

- 1) сформировать обучающий набор данных;
- 2) обучить нейросетевые модели;
- 3) провести эксперименты и выявить лучшую модель;
- 4) разработать веб-приложение для использования модели;
- 5) провести тестирование веб-приложения.

Структура и содержание работы

Работа состоит из введения, трех глав, заключения и списка литературы. Объем работы составляет 34 страницы, объем списка литературы – 24 источника.

В первой главе описывается анализ предметной области – обзор аналогичных проектов, а также используемые инструменты для разработки.

Вторая глава посвящена процессу обучения моделей машинного обучения. В этой главе описывается подбор данных для обучения, непосредственно обучение нейросетевой модели, проведение экспериментов и варианты улучшения модели.

В третьей главе представлен процесс проектирования и реализации веб-приложения для использования моделей машинного обучения.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Обзор аналогичных проектов

Развитие технологий компьютерного зрения и глубокого обучения позволило создать эффективные системы детекции дорожных объектов, которые важны для обеспечения безопасности современных транспортных средств. Использование алгоритмов в реальном времени, позволяет обрабатывать видеопотоки с камер, быстро и точно распознавая пешеходов, транспортные средства, дорожные знаки и разметку. Ниже приведены примеры крупных компаний и проектов, активно использующих данные технологии.

Компания Tesla интегрирует детекцию объектов в свою систему автопилота, которая объединяет данные с множества камер и датчиков. Алгоритмы, основанные на нейросетях, помогают в реальном времени определять расположение объектов, что позволяет автомобилям корректировать траекторию, избегать столкновений и принимать решения на дороге. Высокая скорость обработки и адаптивность системы делают Tesla одним из лидеров в области автономного вождения [11]. Демонстрация работы детекции в автомобилях Tesla представлена на рисунке 2.

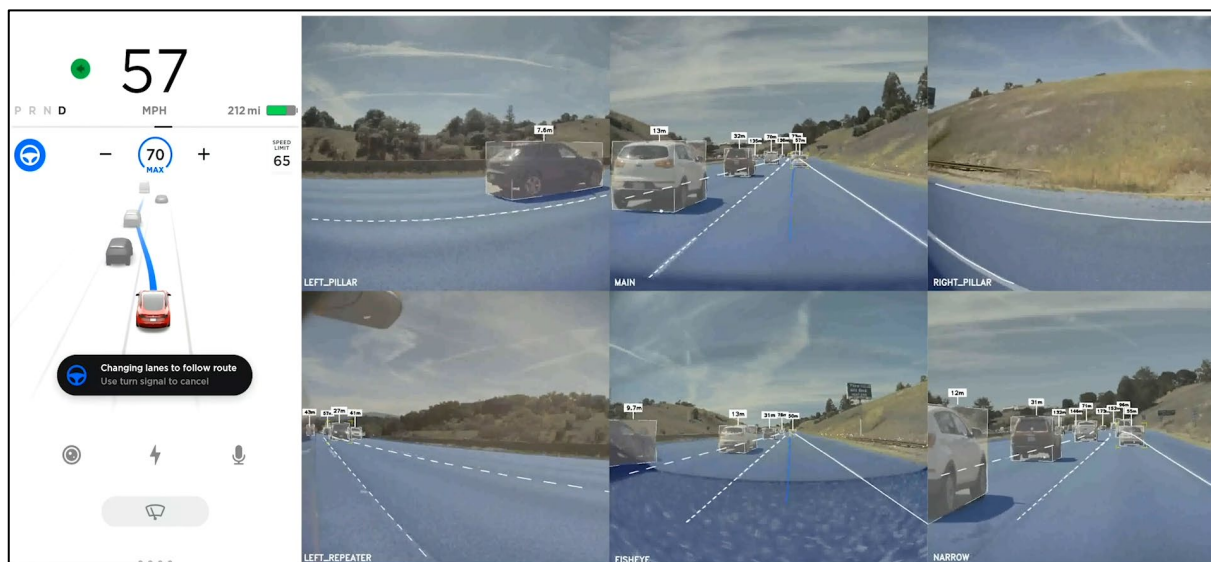


Рисунок 2 – Демонстрация детекции Tesla

Также существует Mobileye – израильская хай-тек компания, разрабатывающая развитые системы помощи водителю (ADAS), приобретенная

компанией Intel в 2017 году. Компания специализируется на системах помощи водителю (ADAS), используя комбинацию алгоритмов компьютерного зрения и искусственного интеллекта для обнаружения дорожных объектов. Технология Mobileye интегрируется в автомобили множества производителей по всему миру, обеспечивая надежное распознавание пешеходов, других транспортных средств и дорожных знаков. [12].

Помимо этого, существуют небольшие проекты с открытым исходным кодом, размещенные в публичных репозиториях. Например, проект «Road Object Detection» [14] использует для обучения набор данных BDD100K, один из крупнейших дорожных наборов данных, содержащий 100 тысяч видео [16]. Этот проект реализован с помощью YOLOv7.

Или, например, проект «Traffic and Road Signs Object Detection Project» [15], который реализован с помощью EfficientDet. Он использует собственный обучающий набор данных, который был сформирован с помощью библиотеки для сбора изображений из интернета «jmd_imagescraper». Всего было собрано около 1 000 изображений.

1.2. Анализ существующих решений

Нейросетевые модели могут иметь разнообразные архитектуры, зависящие от решаемой задачи: сверточные сети для изображений, рекуррентные – для обработки последовательностей и так далее. При выборе архитектуры важно учитывать вычислительные ресурсы, требования к точности и скорость обработки.

В данном случае работа подразумевает работу с изображениями, так что лучшим выбором будут сверточные сети. Далее следует разобраться конкретно с задачей детекции. И для ее решения существует несколько подходов.

Простой подход заключается в том, что объекту задается изображение-образец, которым сканируется изображение, вычисляя корреляцию образца с каждым фрагментом изображения [6].

Двухстадийные детекторы отличаются тем, что сначала выделяются области-кандидаты, а затем уточняют их классификацию и границы, обеспечивая высокую точность, но меньшую скорость (R-CNN, Fast R-CNN, Faster R-CNN, Mask R-CNN) [2].

Одностадийные детекторы выполняют задачу за один проход, одновременно предсказывая классы и границы объектов, что делает их быстрыми (RetinaNet, EfficientDet, YOLO, SSD) [2].

Различные существующие решения применяют разные архитектуры нейронных сетей в соответствии с применяемым подходом. Далее представлены некоторые архитектуры нейросетей для детекции объектов.

1. R-CNN (Regions with CNN features) – объединяет методы извлечения признаков на основе сверточных нейронных сетей (CNN) с традиционными подходами к обнаружению объектов.

2. YOLO (You Only Look Once) [18] – решает задачу детекции объектов за одно обращение к сети. Изображение делится на сетку, где каждая ячейка отвечает за предсказание ограничительных рамок и вероятностей классов объектов, находящихся в пределах этой ячейки.

3. SSD (Single Shot Detector) – генерирует шаблонные ограничивающие рамки с различным соотношением сторон и различными масштабами для карты признаков изображения. Затем нейронная сеть предсказывает вероятности обнаружения целевых классов в шаблонных ограничивающих рамках и уточняет их координаты. Основным отличием от YOLO является использование карт признаков с разных слоев, что повышает точность детектирования объектов разного размера.

4. EfficientDet – состоит из EfficientNet в качестве основы и использует пирамиду признаков BiFPN (Bidirectional Feature Pyramid Network) для объединения признаков с различных уровней иерархии, что позволяет улучшить детекцию объектов разных размеров.

Для обучения нейросети нужен набор размеченных данных. Разметка одна из важнейших задач при обучении нейросети, и от качества данных

зависит работа модели. Существует различное программное обеспечение для разметки данных.

1. VGG Image Annotator (VIA) – это простое и автономное программное обеспечение, для ручного аннотирования изображений, аудио и видео, разработано в Оксфордском университете [8].

2. Roboflow – это мощная платформа, предназначенная для создания и управления наборами данных для задач компьютерного зрения, включая аннотирование изображений [9].

3. Computer Vision Annotation Tool (CVAT) – это веб-инструмент, разработанный компанией Intel для аннотирования изображений и видео с открытым исходным кодом, используемый для маркировки данных для алгоритмов компьютерного зрения [10].

Таким образом, для решения поставленной задачи были выбраны проект Ultralytics YOLO для обучения нейросетевой модели и вычисления метрик. Модели YOLO демонстрируют хорошую скорость и точность. Также Ultralytics предоставляют Python-библиотеку YOLO, для удобной работы с моделями и расчета метрик.

Для формирования набора данных был выбран инструмент CVAT так как это эффективный инструмент для разметки, с открытым исходным кодом и его можно удобно развернуть на персональном компьютере с помощью Docker.

1.3. Метрики качества

Метрики качества – это числовые показатели, которые позволяют оценить работу нейросети (или любого другого алгоритма машинного обучения) на валидационной или тестовой выборке [21]. Выбор метрики зависит от типа задачи (классификация, регрессия, сегментация и т. д.) и от бизнес-целей. Ниже будут рассмотрены ключевые метрики для задач детекции.

Обозначим, количество правильно предсказанных объектов с положительной меткой – TP, количество неправильно предсказанных объектов с

положительной меткой – FP, количество правильно предсказанных объектов с отрицательной меткой – TN, количество неправильно предсказанных объектов с отрицательной меткой – FN. По результатам предсказаний можно построить матрицу ошибок (таблица 1).

Таблица 1 – Матрица ошибок

	Предсказано Р	Предсказано N
Истинно Р	TP (True Positive)	FN (False Negative)
Истинно N	FP (False Positive)	TN (True Negative)

Accuracy (точность)

Точность показывает долю правильных предсказаний модели (и позитивных, и негативных) от общего числа случаев. Считается по формуле (1):

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad (1)$$

Данная метрика хорошо отражает качество модели при сбалансированных классах. Используется как простой общий индикатор обучения модели.

Precision (прецизионность)

Прецизионность показывает долю истинно положительных предсказаний среди всех предсказаний класса «положительный». Считается по формуле (2):

$$Precision = \frac{TP}{TP + FP}. \quad (2)$$

Данная метрика важна, когда необходимо минимизировать ложные срабатывания. Например, при детекции объектов, чтобы каждое «положительное» предсказание было надежным.

Recall (полнота)

Полнота отражает, какую часть всех реальных объектов нужного класса модель сумела найти. Считается по формуле (3):

$$Recall = \frac{TP}{TP + FN}. \quad (3)$$

Важна, когда критично не упустить «положительные» объекты (когда стоимость пропущенного случая (FN) высока).

Average Precision

Метрика Average Precision (AP) одна из ключевых метрик в задача детекции и информационного поиска, которая измеряет качество ранжирования предсказаний. Измеряет площадь под кривой Precision-Recall. Можно посчитать по формуле (4):

$$AP = \int_0^1 Precision(r)dr. \quad (4)$$

Intersection over Union

IoU – это мера перекрытия между предсказанным и истинным (ground-truth) ограничивающими прямоугольниками (bounding boxes). Она показывает, какая доля объема пересечения двух прямоугольников приходится на их объединенный объем. Можно представить в виде формулы (5):

$$IoU(B_{pred}, B_{gt}) = \frac{area(B_{pred} \cap B_{gt})}{area(B_{pred} \cup B_{gt})}. \quad (5)$$

mAP50 (mean Average Precision при IoU=0,5)

Данная метрика показывает среднюю точность обнаружения объектов при пороге IoU=0,50. Для каждого класса вычисляется Average Precision при заданном IoU. Можно посчитать по формулам (6, 7):

$$AP_c = \int_0^1 Precision_c(r)dr, \quad (6)$$

$$mAP_{50} = \frac{1}{C} \sum_{c=0}^C AP_c(IoU = 0,5), \quad (7)$$

где C – число классов, индекс c – означает подсчет для конкретного класса.

Метрика mAP50 – стандартная метрика в задачах детекции. Она показывает, насколько точно модель локализует объекты при пороге перекрытия 0,5. Этот порог считается «слабым», так как он требует лишь умерен-

ного совпадения между предсказанным и истинным ограничивающими прямоугольниками.

mAP50-95 (mean Average Precision при IoU от 0,50 до 0,95)

Данная метрика показывает среднюю точность обнаружения объектов по диапазону IoU[0,5; 0,95] с шагом 0,05. Можно посчитать по формуле (8):

$$mAP_{50} = \frac{1}{C} \sum_{c=0}^C \left(\frac{1}{10} \sum_{t=0}^9 AP_c(IoU = 0,5 + 0,05t) \right). \quad (8)$$

Метрика mAP50–95 более полно отражает качество детекции на разной строгости. Она учитывает как «легкие» случаи (IoU=0.50), так и «сложные» (IoU=0.95) и дает комплексную оценку.

В реализации Ultralytics YOLO эти метрики рассчитываются автоматически после каждой эпохи на валидационном наборе. Мониторинг этих метрик позволяет понять, как происходит обучение модели и отследить появление переобучения.

2. НЕЙРОСЕТЕВАЯ МОДЕЛЬ ДЕТЕКЦИИ ОБЪЕКТОВ

2.1. Обучающий набор данных

Наборы данных или же датасеты представляют собой тщательно собранные и аннотированные коллекции изображений или видео, которые служат основой для обучения и оценки алгоритмов компьютерного зрения.

Датасеты играют решающую роль в обучении моделей компьютерного зрения, позволяя им распознавать объекты, понимать сцены и делать прогнозы на основе визуальной информации. Качество и разнообразие набора данных напрямую влияют на производительность модели: чем более разнообразен и точно аннотирован датасет, тем лучше модель будет работать в реальных условиях.

Для эффективного обучения и оценки моделей компьютерного зрения наборы данных делятся на три основные части: обучающую, валидационную и тестовую выборки [21].

Обучающая часть датасета используется для обучения модели. Обучающая выборка должна быть достаточно большой и разнообразной, чтобы модель могла учиться распознавать различные закономерности и объекты. В каждой эпохе обучения модель обрабатывает одни и те же данные, совершенствуя свои навыки распознавания [21].

Валидационная выборка используется для оценки производительности модели во время обучения. Она помогает в настройке гиперпараметров модели и предотвращает переобучение, когда модель становится слишком специализированной на обучающих данных и не дает корректные ответы для новых, не виденных ранее данных [21].

Тестовая выборка используется для окончательной оценки модели после завершения обучения. Она дает объективную оценку того, насколько хорошо модель работает на незнакомых данных, что важно для понимания ее реальной эффективности [21].

Для данной работы за основу был взят набор данных «Road DETECTION - IMGs & Labels» [17] с платформы Kaggle [13]. Платформа

представляет собой открытую площадку, на которой собрана информация по Data Science и Machine learning.

Данный датасет содержит 1 588 изображений и разделен на обучающую, валидационную и тестовую выборки следующим образом: обучающая – 1 263 изображения, валидационная – 158 изображений, тестовая – 167 изображений.

Изображения разделены на 9 классов:

- светофор;
- знак «стоп»;
- знак ограничения скорости;
- знак пешеходного перехода;
- зебра пешеходного перехода;
- пешеход;
- автобус;
- легковой автомобиль;
- грузовой автомобиль.

Для лучшей работы желательно чтобы набор данных был сбалансированным, то есть все классы были представлены примерно в равных пропорциях. Используемый датасет плохо сбалансирован, это видно из диаграммы, представленной на рисунке 3. Однако для поставленной задачи этого будет достаточно. Для начала стоит провести эксперименты и отталкиваться от их результатов.

Также этот набор данных изначально был не полностью размечен, так что его требовалось дополнительно разметить, чтобы избежать ошибок при работе и улучшить качество обучения модели. Для этого использовался инструмент CVAT. Интерфейс программы представлен на рисунке 4.

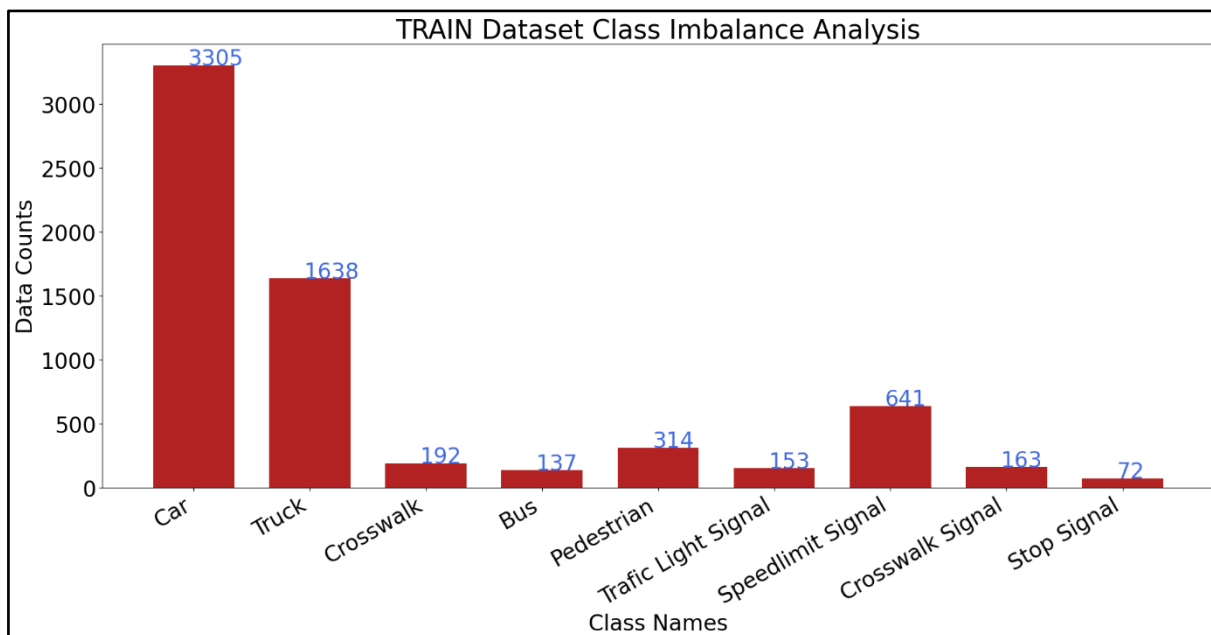


Рисунок 3 – Диаграмма дисбаланса данных

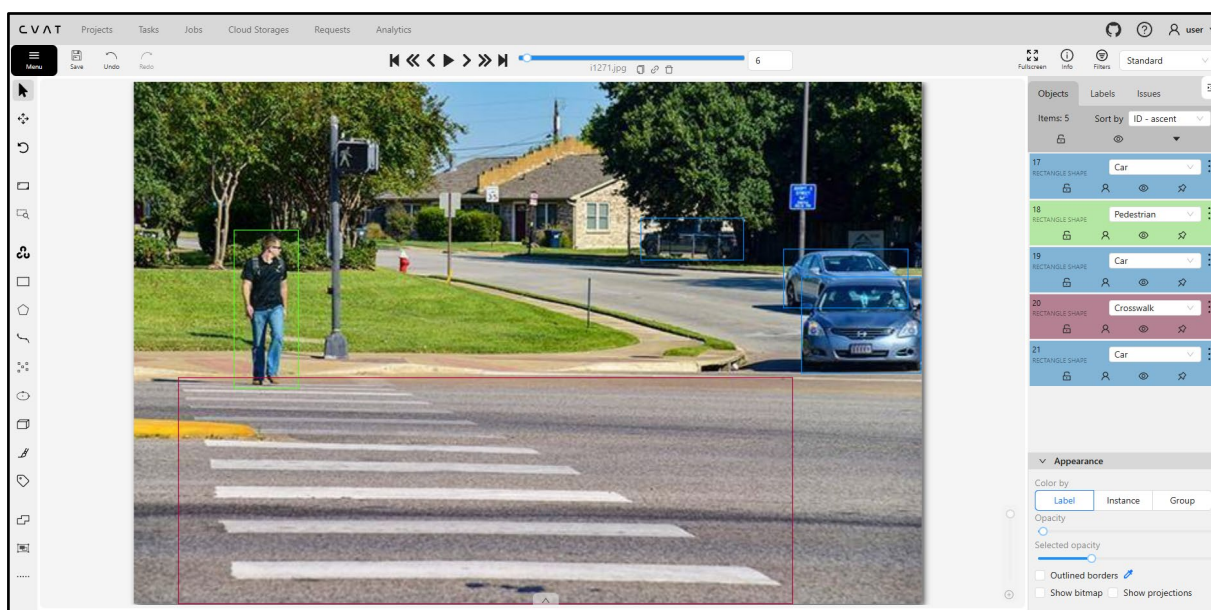


Рисунок 4 – Интерфейс программы CVAT

2.2. Обучение нейросетевых моделей

После того как данные подготовлены необходимо обучить нейросетевую модель. Обучение нейросетевых моделей – это комплексный процесс, который включает в себя подбор архитектуры и гиперпараметров, организацию тренировочного процесса и последующую оптимизацию модели для решения конкретных задач.

Обучение происходит итеративно: модель постепенно корректирует свои веса, минимизируя функцию потерь на обучающих данных. Каждая такая итерация называется эпохой обучения. Параллельно проводится валидация, чтобы отслеживать, как модель справляется с новыми данными, и корректировать гиперпараметры.

После завершения обучения проводится тестирование модели на ранее не виденных данных. Анализируются метрики качества – числовые показатели, которые высчитываются по специальным формулам. Они позволяют оценить работу нейросети (подробная информация по метрикам представлена в разделе 1.3).

Если результаты неудовлетворительны, могут быть внесены изменения: увеличение объема данных, подбор других гиперпараметров или изменение архитектуры. Проект Ultralytics YOLO [18] содержит множество моделей для разных целей и распространяется под свободной лицензией, являясь одним из популярных решений в мире.

YOLO (You Only Look Once) появился как концепция, позволяющая выполнять обнаружение объектов за один проход через сеть. Первоначальная версия продемонстрировала значительный прирост скорости по сравнению с традиционными методами, такими как региональные сверточные нейросети (R-CNN). С тех пор было выпущено несколько версий, каждая из которых улучшала точность, устойчивость и производительность.

У большинства архитектур нейросетей для детекции есть несколько схожих частей. Основной принцип показан на рисунке 5.

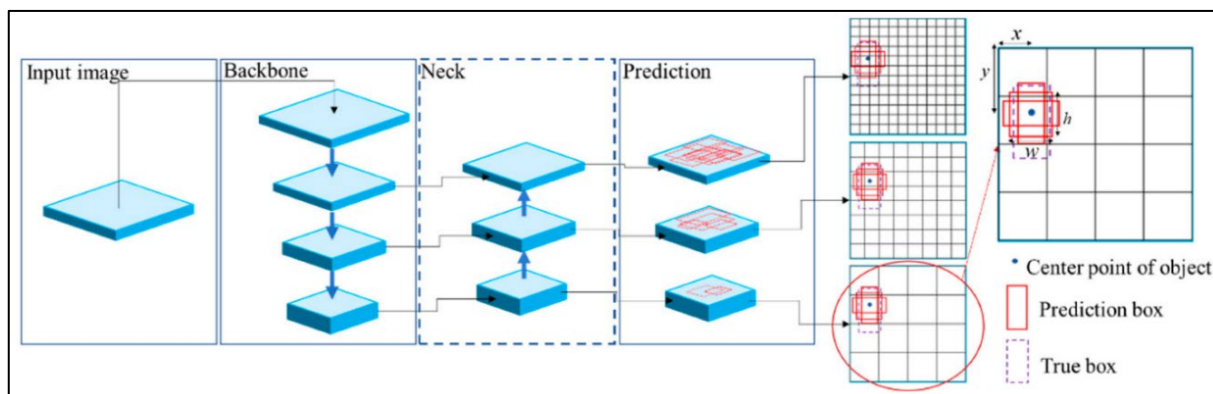


Рисунок 5 – Основа архитектуры

Так, например, основная архитектура YOLO состоит из трех ключевых компонентов: Backbone, Neck, Head.

Backbone – основная часть сети, отвечает за первичную обработку входного изображения. Ее задача – выделить пространственные и семантические признаки, которые затем будут использованы для распознавания объектов. Например, в YOLOv5 используется структура New CSP-Darknet⁵³¹, которая разделяет входящий тензор на две части, обрабатывают одну из них через несколько сверточных блоков, а затем объединяют с неизменной частью.

Backbone состоит из множества сверточных слоев и блоков, каждый из которых постепенно уменьшает размер пространственных карт признаков и увеличивает количество каналов. Это позволяет сети переходить от низкоуровневых признаков (например, краев и текстур) к высокоуровневым (формы, композиции).

Neck служит связующим звеном между извлечением признаков (Backbone) и финальной стадией предсказания (Head). Его основная задача – объединить и адаптировать признаки, полученные на разных этапах backbone, чтобы сделать их максимально информативными для детекции объектов.

Head – это последний этап, на котором агрегированные признаки преобразуются в конкретные результаты. Здесь производится непосредственное прогнозирование местоположения объектов, их классов и уверенности.

Конкретно YOLO на последнем этапе использует подход плотное предсказание (Dense Prediction). Основная суть такого подхода заключается в том, что модель на каждом шаге «смотрит» на все позиции входного изображения и пытается выдать там детекцию. В зависимости от версии YOLO детали работы немного меняются.

Так как YOLO имеет множество версий и моделей, так что необходимо выбрать наиболее подходящие для данной работы.

Была выбрана YOLO11, так как является последней доступной на данный момент версией. Для сравнения также были выбраны YOLOv5 и YOLOv8 как наиболее популярные. Эти семейства моделей существуют уже несколько лет и зарекомендовали себя как эффективные решения для задачи детекции.

Каждая версия YOLO предоставляет несколько моделей разных размеров: nano, small, medium, large, extra large.

В таблице 2 представлены параметры использования моделей YOLOv5, обученных на наборе данных COCO (Common Objects in Context) и включающих 80 предварительно обученных классов. Эти данные представлены компанией Ultralytics на официальном сайте и позволяют пользователям оценить эффективность моделей и сравнить их друг с другом [18].

Таблица 2 – Параметры моделей YOLOv5

Модель	Слои	mAPval 50-95	Время обучения CPU, мс	Время обу- чения GPU A100, мс	Пара- метры, млн	FLOPs, млрд
yolov5nu	122	34,3	73,6	1,06	2,6	7,7
yolov5su	213	43,0	120,7	1,27	9,1	24,0
yolov5mu	290	49,0	233,9	1,86	25,1	64,2
yolov5lu	315	52,2	408,4	2,50	53,2	135,0
yolov5xu	315	53,2	763,2	3,81	97,2	246,4

В таблице 3 представлены аналогичные параметры использования всех моделей семейства YOLOv8.

Таблица 3 – Параметры моделей YOLOv8

Модель	Слои	mAPval 50-95	Время обучения CPU, мс	Время обу- чения GPU A100, мс	Пара- метры, млн	FLOPs, млрд
YOLOv8n	225	37,3	80,4	0,99	3,2	8,7
YOLOv8s	225	44,9	128,4	1,20	11,2	28,6
YOLOv8m	295	50,2	234,7	1,83	25,9	78,9
YOLOv8l	365	52,9	375,2	2,39	43,7	165,2
YOLOv8x	365	53,9	479,1	3,53	68,2	257,8

В таблице 4 представлены аналогичные параметры использования всех моделей семейства YOLO11.

Таблица 4 – Параметры моделей YOLO11

Модель	Слои	mAPval 50-95	Время обучения CPU, мс	Время обу- чения GPU T4, мс	Пара- метры, млн	FLOPs, млрд
YOLO11n	319	39,5	56,1	1,5	2,6	6,5
YOLO11s	319	47,0	90,0	2,5	9,4	21,5
YOLO11m	409	51,5	183,2	4,7	20,1	68,0
YOLO11l	631	53,4	238,6	6,2	25,3	86,9
YOLO11x	631	54,7	462,8	11,3	56,9	194,9

Nano – наименьшая модель, предоставляет наилучшую производительность, подходит для устройств с небольшой вычислительной мощностью. В свою очередь Extra Large – наибольшая модель, показывает наивысшую точность, но низкую скорость.

Для работы была выбрана модель Small, так как она предоставляет оптимальное соотношение качества и скорости для поставленной задачи.

2.3. Эксперименты

На данном этапе будут обучены несколько нейросетевых моделей, и с помощью метрик будет определена лучшая.

Обучение осуществлялось посредством облачных вычислений в среде Google Colaboratory. Она предоставляет удобный формат работы в виде блокнотов Jupyter Notebook. А также дает бесплатный доступ к вычислительным мощностям графического процессора, что значительно поднимет скорость обучения нейросетевых моделей.

Перед обучением необходимо задать параметры обучения. Ultralytics YOLO предоставляет возможность тонко настраивать нейросеть посредством множества параметров. Далее представлены некоторые часто настраиваемые параметры [18].

Скорость обучения (lr0) – определяет размер шага на каждой итерации при движении к минимуму в функции потерь.

Размер батча (batch) – количество изображений, обрабатываемых одновременно за один проход нейросети.

Оптимизатор (optimizer) – алгоритм, отвечающий за подбор таких параметров модели, как веса и смещение, стремясь минимизировать функцию потерь. При работе с Ultralytics YOLO при выборе оптимизатора доступны следующие варианты: SGD, Adam, Adamax, AdamW, NAdam, RAdam, RMSProp, auto. В ходе работы использовался параметр auto. В таком случае оптимизатор выбирается автоматически на основе размера данных и архитектуры.

В рамках данной работы эти параметры, как и многие другие не будут изменяться, так как по умолчанию уже указаны оптимальные и универсальные значения. Изменяться будет только количество эпох. Эпоха – это один полный проход нейронной сети по всему обучающему набору данных.

При тестировании было обнаружено, что модели достигают наивысшей точности на промежутке от 15 до 20 эпохи и дальнейшее обучение не приводит к улучшению моделей. Поэтому все модели будут обучаться на протяжении 20 эпох. Исходя из результатов обучения, можно будет определить какая версия лучше справляется.

Использовался графический процессор NVIDIA T4 и при обучении было задействовано до 4 гигабайт оперативной памяти и до 5 гигабайт видеопамяти. Каждая из моделей занимает приблизительно 23 мегабайта ПЗУ. Само обучение длилось приблизительно 20 минут для каждой модели.

Для сравнения использовались наиболее показательные метрики mAP50 и mAP50-95. Итоговый результат, показывающий точность обучения нейросетевых моделей, представлен в таблице 5.

Таблица 5 – Метрики обученных моделей

Модель	mAP50	mAP50-95
YOLOv5	0,829	0,615
YOLOv8	0,834	0,626
YOLO11	0,831	0,611

Исходя из результатов, можно сделать вывод, что лучше всего себя проявила модель семейства YOLOv8. В таблице 6 представлены метрики модели YOLOv8 для каждого класса.

Таблица 6 – Метрики модели YOLOv8 для каждого класса.

Class	Images	Instances	P	R	mAP50	mAP50-95
all	158	852	0,837	0,818	0,834	0,626
Traffic Light Signal	9	18	0,753	0,778	0,772	0,5
Stop Signal	8	8	0,736	1	0,926	0,865
Speedlimit Signal	67	71	0,984	1	0,995	0,902
Crosswalk Signal	18	22	1	0,858	0,888	0,731
Crosswalk	25	26	0,829	0,808	0,84	0,601
Pedestrian	14	35	0,822	0,914	0,943	0,654
Bus	13	14	0,729	0,429	0,424	0,331
Car	62	441	0,843	0,751	0,838	0,486
Truck	44	217	0,841	0,826	0,879	0,562

Для удобства понимания и интерпретации данных ниже приводится описание терминов, используемых в таблице.

1. Class – название класса.
2. Images – число изображений с данным классом.
3. Instances – число объектов данного класса на всех изображениях.
4. P – метрика Precision.
5. R – метрика Recall.
6. mAP50 – метрика mAP50.
7. mAP50-95 – метрика mAP50-95.

На рисунке 6 представлена матрица ошибок для лучшей полученной нейросетевой модели. Матрица ошибок (confusion matrix) – это квадратная таблица, которая используется для оценки эффективности моделей классификации и детекции. В этой таблице столбцы соответствуют истинным классам, а строки – предсказанным моделью классам.

Матрица ошибок позволяет оценить общую точность модели, а также понять, какие именно ошибки чаще всего совершает классификатор, что важно для дальнейшей доработки модели и анализа ее слабых мест.

На главной диагонали матрицы отображено количество верных предсказаний. Вне диагонали находятся ошибки классификации – случаи, когда модель спутала один класс с другим.

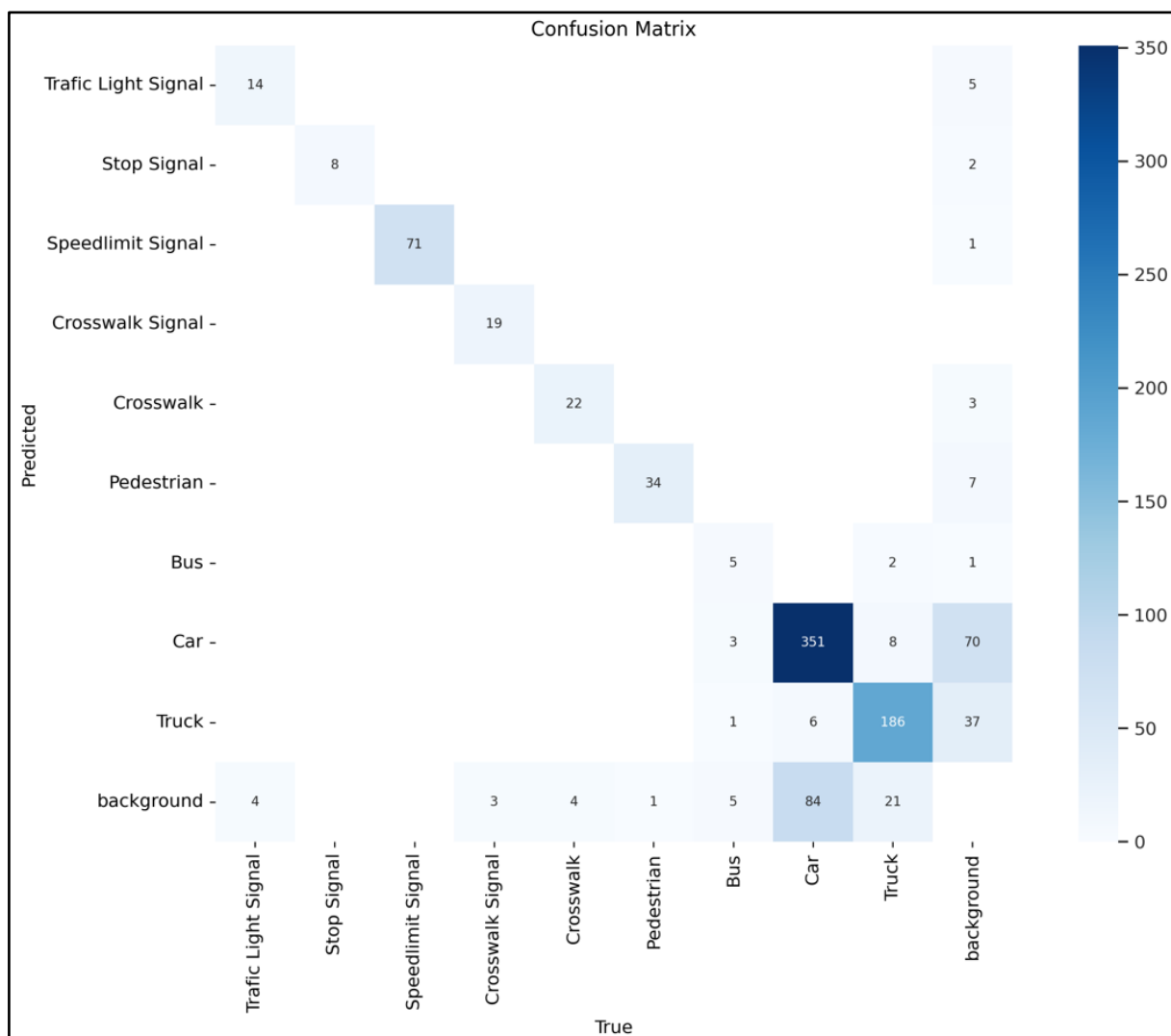


Рисунок 6 –Матрица ошибок

Анализируя полученные результаты, было обнаружено, что модель демонстрирует приемлемый уровень точности на всех объектах кроме автобусов. Поэтому было принято решение расширить набор данных и добавить дополнительные изображения с автобусами для повышения точности.

После обновления датасета необходимо заново обучить нейросетевую модель. Так как в первом эксперименте лучше всех себя показала модель YOLOv8, она также использовалась для обучения на обновленном наборе данных. В таблице 7 представлены метрики модели YOLOv8, обученной на новом наборе данных.

Таблица 7 – Метрики новой модели YOLOv8 для каждого класса

Class	Images	Instances	P	R	mAP50	mAP50-95
all	193	929	0,852	0,831	0,884	0,662
Traffic Light Signal	11	23	0,795	0,652	0,755	0,458
Stop Signal	8	8	0,759	1	0,995	0,952
Speedlimit Signal	67	71	0,974	1	0,995	0,899
Crosswalk Signal	18	22	0,857	0,814	0,884	0,752
Crosswalk	25	26	0,810	0,821	0,902	0,613
Pedestrian	18	42	0,898	0,833	0,911	0,558
Bus	47	65	0,893	0,771	0,838	0,702
Car	68	455	0,869	0,732	0,834	0,471
Truck	44	217	0,812	0,858	0,888	0,552

Из таблицы видно, что точность распознавания автобусов стала выше, за счет этого и общая средняя точность возросла.

Ниже представлен пример входных и выходных данных. На рисунке 7 представлено изображение дорожного перекрестка.



Рисунок 7 – Изображение до обработки

На рисунке 8 представлено то же изображение, но обработанной нейросетевой моделью. На нем все обнаруженные объекты помечены ограничивающими рамками.

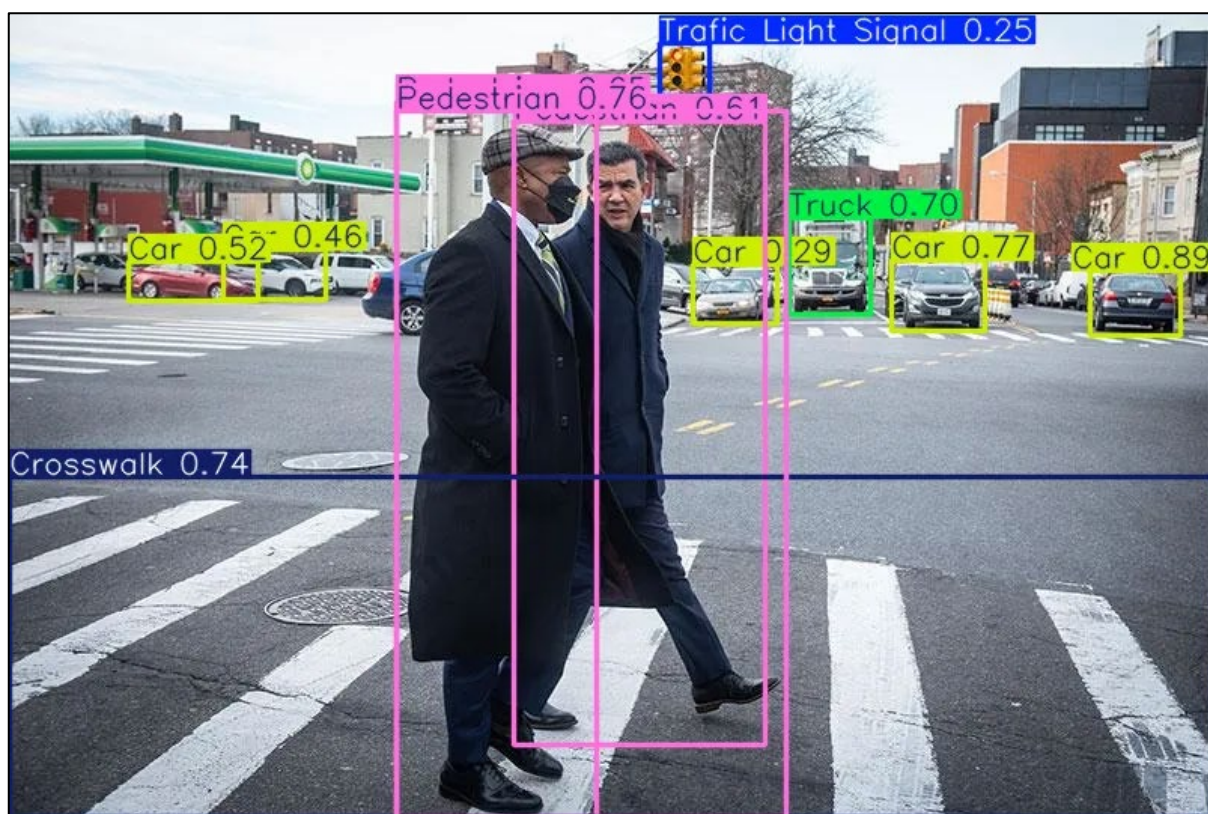


Рисунок 8 – Изображение после обработки

На обработанном изображении видно, что большую часть хорошо видимых объектов модель распознала и верно классифицировала. Однако можно заметить и пропущенные объекты, например, несколько автомобилей на заправочной станции в левой части изображения и две разметки пешеходного перехода в левой и правой частях изображения.

Таким образом в ходе работы были обучена нейросетевая модель, демонстрирующая среднюю точность выше 88% по метрике mAP50. Для дальнейшего улучшения нейросетевой модели наиболее верным способом будет увеличение набора данных новыми изображениями.

3. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

3.1. Проектирование

В ходе работы были сформулированы следующие функциональные требования, которые обеспечат широкие возможности для пользователей веб-приложения.

1. Пользователи должны иметь возможность выбирать желаемую нейросетевую модель из списка.
2. Пользователи должны иметь возможность загружать собственные нейросетевые модели детекции.
3. Приложение должно обрабатывать изображения и видео.

Нефункциональным требованием для разрабатываемого приложения является разработка приложения на языке программирования Python.

Используемая в приложении нейросетевая модель должна быть основана на модели Ultralytics YOLO.

На рисунке 9 представлена диаграмма компонентов, демонстрирующая архитектуру веб-приложения. По этой диаграмме видно, какие основные компоненты используются в системе.

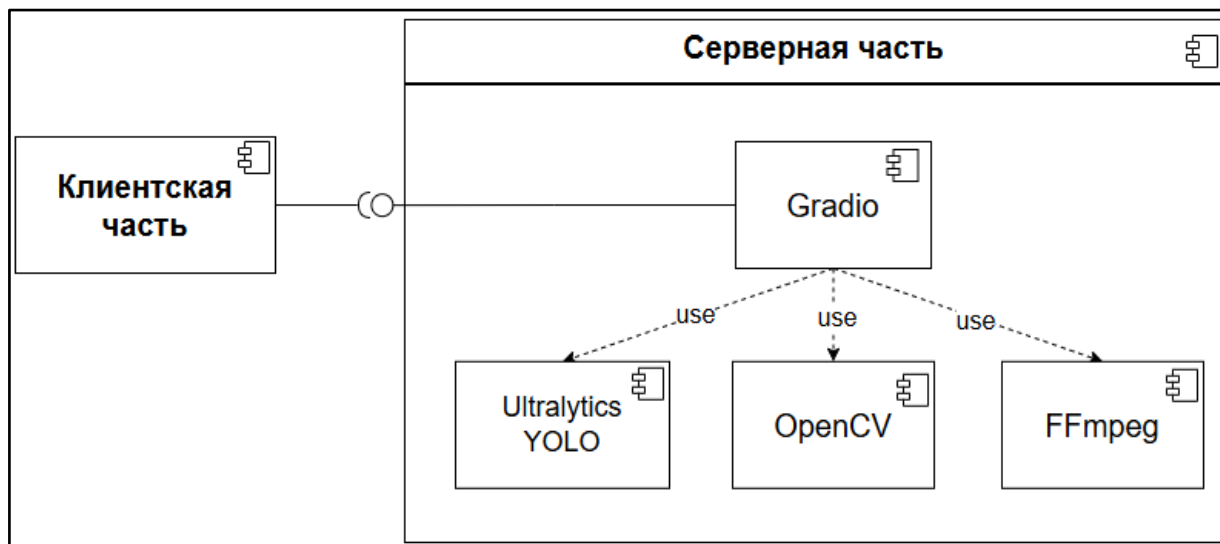


Рисунок 9 – Диаграмма компонентов

На рисунке 10 представлена последовательность действий при обработке видеофайла, отображающая алгоритм работы веб-приложения. Актором, инициирующим действия в системе, выступает пользователь. Для обработки видео необходимо разделить его на кадры, и обработать каждый отдельно. После чего собрать результирующее видео и вернуть его пользователю.

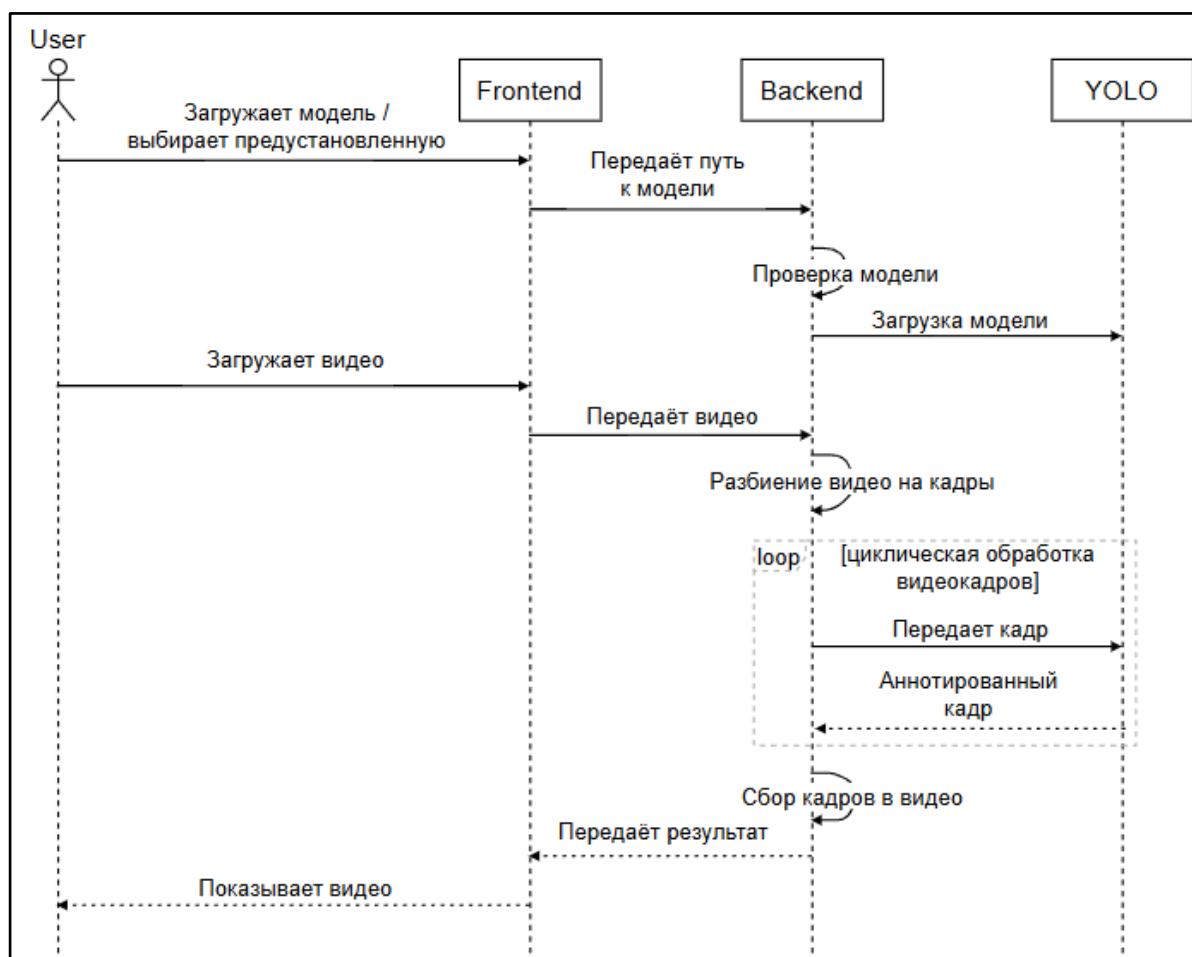


Рисунок 10 – Обработка видео

Таким образом было спроектировано веб-приложение для использования нейросетевых моделей детекции с учетом различных типов входных данных. Сформулированы функциональные и нефункциональные требования к приложению. Также была построена диаграмма компонентов системы. Такой подход способствует более точному пониманию логики работы системы, упрощает этапы разработки и тестирования.

3.2. Серверная часть приложения

Серверная часть или бэкенд [23] отвечает за основной функционал приложения. Он получает запросы от фронтенда, выполняет бизнес-логику, взаимодействует с базами данных или внешними сервисами, вычисляет результаты и возвращает их в удобном формате. Бэкенд занят проверкой корректности входных данных, авторизацией, безопасностью, хранением и преобразованием информации, запуском тяжелых вычислений.

Для разработки был выбран язык программирования Python для легкой интеграции некоторых частей кода, написанного на этапе обучения и тестирования нейросетевых моделей.

В основе бэкенда лежит набор Python-скриптов, отвечающих за управление нейросетевыми моделями, выполнение детекции на изображениях и видео, а также за сохранение результатов. При старте программы автоматически создаются папки для предустановленных и загруженных пользователем моделей, а также для вывода результатов. Это позволяет централизованно хранить все необходимые файлы и контролировать их доступность. Для обнаружения объектов используется библиотека Ultralytics YOLO. С ее помощью можно использовать обученные модели YOLO для детекции на изображениях.

Обработка изображений и видео построена по следующему принципу: на вход подаются файлы, вызывается функция детекции, которая инициализирует модель и проходит по каждой картинке. Для изображений создается аннотированное изображение с рамками объектов, которое сохраняется на диск с уникальным именем, включающим временную метку.

При работе с видео, его сначала необходимо разбить на кадры. Затем кадры читаются и аннотируются. В конце все кадры соединяются в одно видео. Для корректной конвертации видео используется утилита FFmpeg [24].

Весь процесс снабжен подробным логированием от момента загрузки модели до окончания сохранения результатов. Любые ошибки – будь то неправильное расширение загружаемого файла модели или сбой инференса –

обрабатываются локально через функцию `print` и выводятся в интерфейсе через методы `Gradio` (`gr.Warning`, `gr.Error`), чтобы уведомить пользователя о проблемах.

В библиотеке YOLO уже реализована функция обработки изображений. В результате работы этой функции получается изображение с отрисованными ограничивающими рамками. Также подписаны предполагаемые класс объектов и точность определения.

3.3. Клиентская часть приложения

Клиентская часть или фронтенд [23] отвечает за пользовательский интерфейс. В браузере или в десктопном приложении фронтенд отвечает за отображение элементов управления, визуализацию данных, получение от пользователя ввода и отправку его на сервер. В нем отображаются кнопки, поля для ввода данных или загрузки файлов, прогресс-бары. Фронтенд обрабатывает нажатия, следит за изменением состояния интерфейса, показывает результаты работы бэкенда и уведомляет в случае ошибок.

В данном случае клиентская часть представляет собой браузер, отображающий страницу веб-приложения. Веб-страница приложения представляет собой интерфейс программы, который видит пользователь.

Для создания интерфейса использовалась Python библиотека `Gradio`, позволяющая без знания других языков программирования или разметки реализовать веб-интерфейс для приложения. Библиотека `Gradio` [19] часто используется для при разработке приложений на основе нейронных сетей.

Например, на Hugging Face [20], платформе для разработки и распространения ИИ-инструментов, все мини-приложения для работы с нейросетями реализованы с помощью библиотеки `Gradio`.

`Gradio` создает легкий веб-сервер и генерирует HTML/CSS/JS-интерфейс на основе описанных в Python компонентов. Когда пользователь загружает файл или нажимает кнопку, данные автоматически отправляются на сервер, где вызываются привязанные функции, а их

результаты возвращаются обратно и отображаются в соответствующих виджетах.

В начале страницы размещается блок с заголовком и общей инструкцией, после чего идет выпадающий список, показывающий все доступные нейросетевые модели, рядом расположен виджет загрузки собственной модели. При выборе или загрузке новой модели список автоматически обновляется.

Ниже на странице расположен компонент Tabs, разбивающий функциональность по двум вкладкам. В первой пользователь может загрузить одно или несколько изображений, после чего по нажатию кнопки «Начать обработку изображений» запускается соответствующая функция бэкенда. Вторая вкладка организована схожим образом, но она уже предназначена для обработки видеофайлов. Кнопки запуска обработки связаны с Python-функциями через методы `click()`, а Gradio автоматически сериализует переданные файлы, вызывает функцию на сервере и возвращает результат в интерфейс пользователя. На рисунке 11 представлен скриншот разработанного веб-приложения.

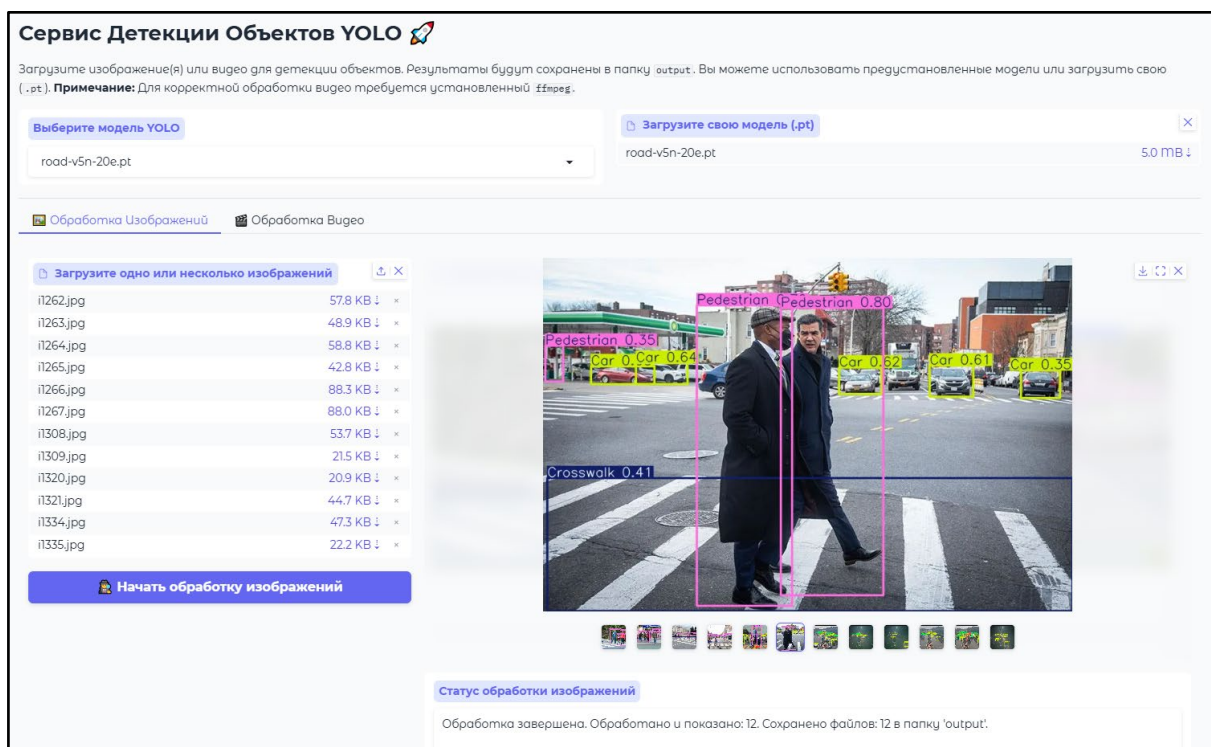


Рисунок 11 – Интерфейс веб-приложения

3.4. Тестирование приложения

Для проверки системы использовалось функциональное тестирование. Это один из самых популярных видов тестирования, который проверяет соответствие функциональности продукта тому, как он был задуман [22]. В таблице 8 представлен протокол функционального тестирования.

Таблица 8 – Функциональное тестирование

№	Название теста	Шаги	Ожидаемый результат	Статус теста
1	Проверка загрузки модели	1. На главной странице нажать кнопку загрузки модели. 2. Выбрать файл модели с расширением «.pt».	В папке <code>uploaded_models</code> появился загружаемый файл.	Пройден
2	Проверка выпадающего меню	1. На главной странице нажать кнопку загрузки модели. 2. Выбрать файл модели с расширением «.pt». 3. На главной странице нажать кнопку выбора модели.	Под кнопкой выбора модели появляется меню с доступными моделями.	Пройден
3	Проверка загрузки и обработки видео	1. На главной странице выбрать пункт «Обработка изображений». 2. Нажать кнопку загрузки изображений. 3. Выбрать один или несколько файлов изображений. 4. Нажать кнопку «Начать обработку».	В соответствующей области появится галерея с результирующими изображениями.	Пройден
4	Проверка загрузки и обработки видео	1. На главной странице выбрать пункт «Обработка видео». 2. Нажать кнопку загрузки видеофайла. 3. Выбрать один видеофайл. 4. Нажать кнопку «Начать обработку».	В соответствующей области появляется плеер для воспроизведения результирующего видео	Пройден

Таким образом было спроектировано и реализовано веб-приложение для использования нейросетевых моделей детекции. Выполнены все функциональные и нефункциональные требования. Функциональное тестирование успешно пройдено.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы бакалавра были обучены нейросетевые модели для детекции дорожных объектов и выбрана модель, показывающая лучший результат. Также было разработано веб-приложение для работы с обученной моделью. При этом были решены следующие задачи:

- 1) найден и дополнительно размечен набор данных;
- 2) дополнен набор данных;
- 3) обучены нейросетевые модели;
- 4) проведены эксперименты и выявлена лучшая модель;
- 5) спроектировано и разработано веб-приложение для использования обученной модели;
- 6) проведено тестирование веб-приложения.

ЛИТЕРАТУРА

1. Нейронные сети в кибербезопасности. [Электронный ресурс] URL: <https://habr.com/ru/articles/587694/> (дата обращения: 13.02.2025 г.).
2. Эволюция архитектур нейросетей в компьютерном зрении. [Электронный ресурс] URL: <https://habr.com/ru/articles/869766/> (дата обращения: 13.02.2025 г.).
3. Новые архитектуры нейросетей. [Электронный ресурс] URL: <https://habr.com/ru/articles/498168/> (дата обращения: 13.02.2025 г.).
4. Шульга Т.Э., Солопекин Д.А. Распознавание дорожных знаков российского образца с использованием нейронных сетей // Вестник АГТУ. Серия: Управление, вычислительная техника и информатика. 2024. №2.
5. СТА – веб-сайт посвященный современной электронике и технологиям автоматизации. [Электронный ресурс] URL: <https://www.cta.ru/articles/soel/2025/se-2025-1/180966/> (дата обращения: 15.02.2025 г.).
6. Онлайн-учебник по машинному обучению и нейросетям. [Электронный ресурс] URL: <https://deepmachinelearning.ru/docs/Neural-networks/Object-detection/Object-detection-task> (дата обращения: 15.02.2025 г.).
7. Precedence Research – платформа статистики. [Электронный ресурс] URL: <https://www.precedenceresearch.com/artificial-intelligence-market> (дата обращения: 20.02.2025 г.).
8. Оксфордский институт робототехники. [Электронный ресурс] URL: <https://www.robots.ox.ac.uk/~vgg/software/via/> (дата обращения: 20.02.2025 г.).
9. Документация Roboflow. [Электронный ресурс] URL: <https://docs.roboflow.com/annotate/annotation-tools> (дата обращения: 20.02.2025 г.).
10. CVAT. [Электронный ресурс] URL: <https://www.cvat.ai> (дата обращения: 20.02.2025 г.).
11. Сайт компании Tesla. [Электронный ресурс] URL: <https://www.tesla.com/support/full-self-driving-computer> (дата обращения: 25.02.2025 г.).

12. Сайт компании Mobileye [Электронный ресурс] URL: <https://www.mobileye.com/solutions/drive/> (дата обращения: 25.02.2025 г.).
13. Платформа Kaggle [Электронный ресурс] URL: <https://www.kaggle.com> (дата обращения: 25.02.2025 г.).
14. YOLOv7 - Road Object Detection. [Электронный ресурс] URL: <https://github.com/pvlakshm/YOLOv7> (дата обращения: 25.02.2025 г.).
15. Traffic and Road Signs Object Detection Project. [Электронный ресурс] URL: <https://github.com/palit-ishan/Traffic-and-Road-Signs-Object-Detection-Project> (дата обращения: 25.02.2025 г.).
16. Исследовательская лаборатория университета Баркли. [Электронный ресурс] URL: <https://bair.berkeley.edu/blog/2018/05/30/bdd/> (дата обращения: 25.02.2025 г.).
17. Набор данных «Road DETECTION - IMGs & Labels». [Электронный ресурс] URL: <https://www.kaggle.com/datasets/princekhunt19/road-detection-imgs-and-labels/data> (дата обращения: 18.02.2025 г.).
18. Документация Ultralytics YOLO. [Электронный ресурс] URL: <https://docs.ultralytics.com/> (дата обращения: 25.02.2025 г.).
19. Gradio. [Электронный ресурс] URL: <https://www.gradio.app> (дата обращения: 10.04.2025 г.).
20. Hugging Face. [Электронный ресурс] URL: <https://huggingface.co/spaces> (дата обращения: 10.04.2025 г.).
21. Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение // пер. с англ. А. А. Слинкина. 2-е издание, испр. М.: ДМК Пресс, 2018. 652 с.: цв. ил.
22. Яндекс Практикум. [Электронный ресурс] URL: <https://practicum.yandex.ru/blog/funkcionalnoe-testirovanie-po/> (дата обращения: 20.04.2025 г.).
23. CODDY – школа программирования. [Электронный ресурс] URL: <https://coddyschool.com/blog/frontend-and-backend-the-concept-and-differences/> (дата обращения: 20.04.2025 г.).
24. FFmpeg. [Электронный ресурс] <https://ffmpeg.org> (дата обращения: 20.04.2025 г.).